

Kubernetes Operator Design Note + Interview Answer

Prepared by: Narendra Bedarkar

Topic: Operator behavior at scale and troubleshooting delayed reconciliation

1) Context

This note explains how a Kubernetes operator works using the `nginx-operator` project and answers the interview scenario:

“If a CR spec change takes around 10 minutes to apply, what would you check?”

The project contains: - CRD/API: `NginxCluster` with `spec.replicas` - Controller: reconciles `NginxCluster` into a `Deployment` and `Service` - Status: updates `status.readyReplicas`

2) Operator Design in This Project

Custom Resource

- `NginxCluster.spec.replicas` defines desired pod count.
- Example input:

```
apiVersion: platform.example.com/v1
kind: NginxCluster
metadata:
  name: nginxcluster-sample
spec:
  replicas: 2
```

Reconciliation Flow

1. Watch `NginxCluster` events.
2. Fetch CR from API server.
3. Reconcile `Deployment` using `CreateOrUpdate`.
4. Reconcile `Service` using `CreateOrUpdate`.
5. Set owner references for garbage collection.
6. Update `NginxCluster.status.readyReplicas`.

Key Properties

- Idempotent reconcile loop (safe to run repeatedly).
- Desired state (`spec`) vs observed state (`status`) model.

- Ownership and lifecycle managed through controller reference.
-

3) How I Would Answer the “10-Minute Delay” Question

Direct Answer

If a `NginxCluster` spec update takes 10 minutes, I debug in layers from controller trigger to workload readiness:

1. **Verify event-to-reconcile latency**
 - Confirm reconcile starts immediately after CR update.
 - Check operator logs for reconcile timestamps.
2. **Measure each reconcile step**
 - Time `Get(CR)`, `CreateOrUpdate(Deployment)`, `CreateOrUpdate(Service)`, `Status().Update`.
 - If these are slow, suspect API pressure, retries, or rate limits.
3. **Validate desired vs actual propagation**
 - Confirm `spec.replicas` change appears quickly in `Deployment.spec.replicas`.
 - If deployment updates quickly but readiness is slow, issue is not controller logic.
4. **Investigate pod rollout bottlenecks**
 - Scheduler/resource pressure (CPU/memory unavailable).
 - Slow image pulls.
 - Readiness probe failures.
 - Node/network instability.
5. **Check status update reliability**
 - Ensure `status.readyReplicas` updates are not failing due to conflicts.
 - Add retries/backoff around status update if required.
6. **Improve observability**
 - Add per-step structured logs and reconcile duration metrics.
 - Add `metav1.Condition` states: `Progressing`, `Available`, `Degraded`.
 - Emit Kubernetes events for major lifecycle transitions.

Why this is strong

- It shows understanding of operator internals, not only Kubernetes operations.
 - It separates control-plane latency from workload readiness latency.
 - It is practical, measurable, and production-oriented.
-

4) 30-Second Interview Version

“In this operator, a CR spec change like `replicas` should trigger reconcile right away. I would first verify reconcile latency in logs, then time each API step: CR fetch, deployment/service reconcile, and status update. If deployment spec updates quickly but pods take long to become ready, the bottleneck is rollout-side (scheduling, image pull, probe failures), not controller logic. I would also add conditions, per-step metrics, and events so delays are diagnosable in minutes.”

5) Suggested Next Improvements for This Repo

1. Add `metav1.Condition` management in status.
2. Add explicit error classification and retry strategy.
3. Add metrics for reconcile duration and failure counts.
4. Add event recording for visibility during rollout.
5. Add tests for delayed readiness and status transitions.